

15. Address Translation

Operating System: Three Easy Pieces

- ▣ Ch. 6: **Limited direct execution** (or LDE): Let the program run directly on the hardware; however, at certain key points in time (say a system call or a timer interrupt), arrange so that the OS gets involved and makes sure the “right” thing happens.

- For Memory virtualizing, we want to take something from limited direct execution (LDE) while introducing **flexibility**.
- Specifically, we'd like for programs to use their address spaces in whatever way they would like, thus making the system easier to program.
- In memory virtualizing, efficiency, and control are attained by **hardware support**.

Address Translation

- ❑ Hardware transforms a **virtual address** into a **physical address**, creating a *beautiful illusion*.
 - ◆ The desired information is stored in a physical address.
- ❑ The OS must get involved at key points to set up the hardware.
 - ◆ The OS must manage memory to judiciously intervene.

Assumptions

- ▣ Stop here and ask the students, what assumptions could be useful.

Assumptions

1. User's address space must be placed contiguously in physical memory.
 2. The size of the address space is not too big; specifically, it is less than the size of physical memory.
 3. Finally, we will also assume that each address space is exactly the same size.
- ▣ Don't worry, we will relax these as we go

Example: Address Translation

▣ C - Language code

```
void func()  
    int x;  
    ...  
    x = x + 3; // this is the line of code we are interested in
```

- ◆ **Load** a value from memory
- ◆ **Increment** it by three
- ◆ **Store** the value back into memory

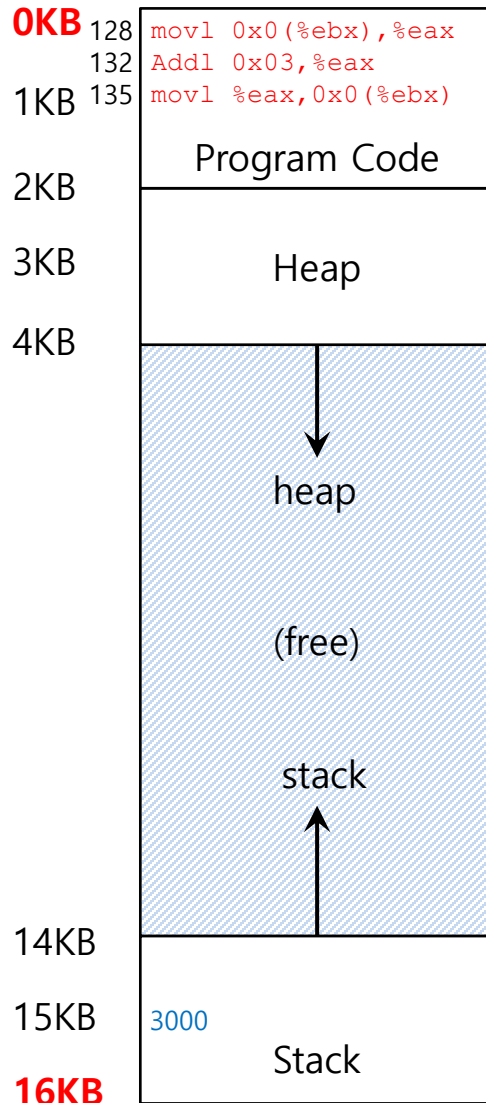
Example: Address Translation(Cont.)

- ❑ In Linux, you have a cool tool called **objdump**
- ❑ Example use: `objdump -d program.out`

```
128 : movl 0x0(%ebx), %eax      ; load 0+ebx into eax
132 : addl $0x03, %eax          ; add 3 to eax register
135 : movl %eax, 0x0(%ebx)       ; store eax back to mem
```

- ❑ Assembly
 - ◆ Presume that the address of 'x' has been place in `ebx` register.
 - ◆ **Load** the value at that address into `eax` register.
 - ◆ **Add** 3 to `eax` register.
 - ◆ **Store** the value in `eax` back into memory.

Example: Address Translation(Cont.)



- Fetch instruction at address 128
- Execute this instruction (load from address 15KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)
- Note: Code is present in the Code space.
- Note: Data is present in Stack space.
- Note: the address starts at **0** and ends at **16KB**.

Relocation Address Space

- ▣ The OS wants to place the process **somewhere else** in physical memory, **not at address 0**.
 - ◆ **Why not?**
 - ◆ The address space starts at address 0.

Dynamic (Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
 - ◆ Set the **base register** a value.

$$\text{physical address} = \text{virtual address} + \text{base}$$

- ◆ Every virtual address must **not be greater than bound** and **negative**.

$$0 \leq \text{virtual address} < \text{bounds}$$

- ◆ We need 2 hardware registers within each CPU: (1) **base**, and (2) **bound** registers.

Dynamic (Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
 - ◆ Set the **base register** a value.

$$\text{physical address} = \text{virtual address} + \text{base}$$

- ◆ Virtual address: 0 KB.
- ◆ Base = 32 KB.
- ◆ Physical address = 32KB + 0 = 32 KB.

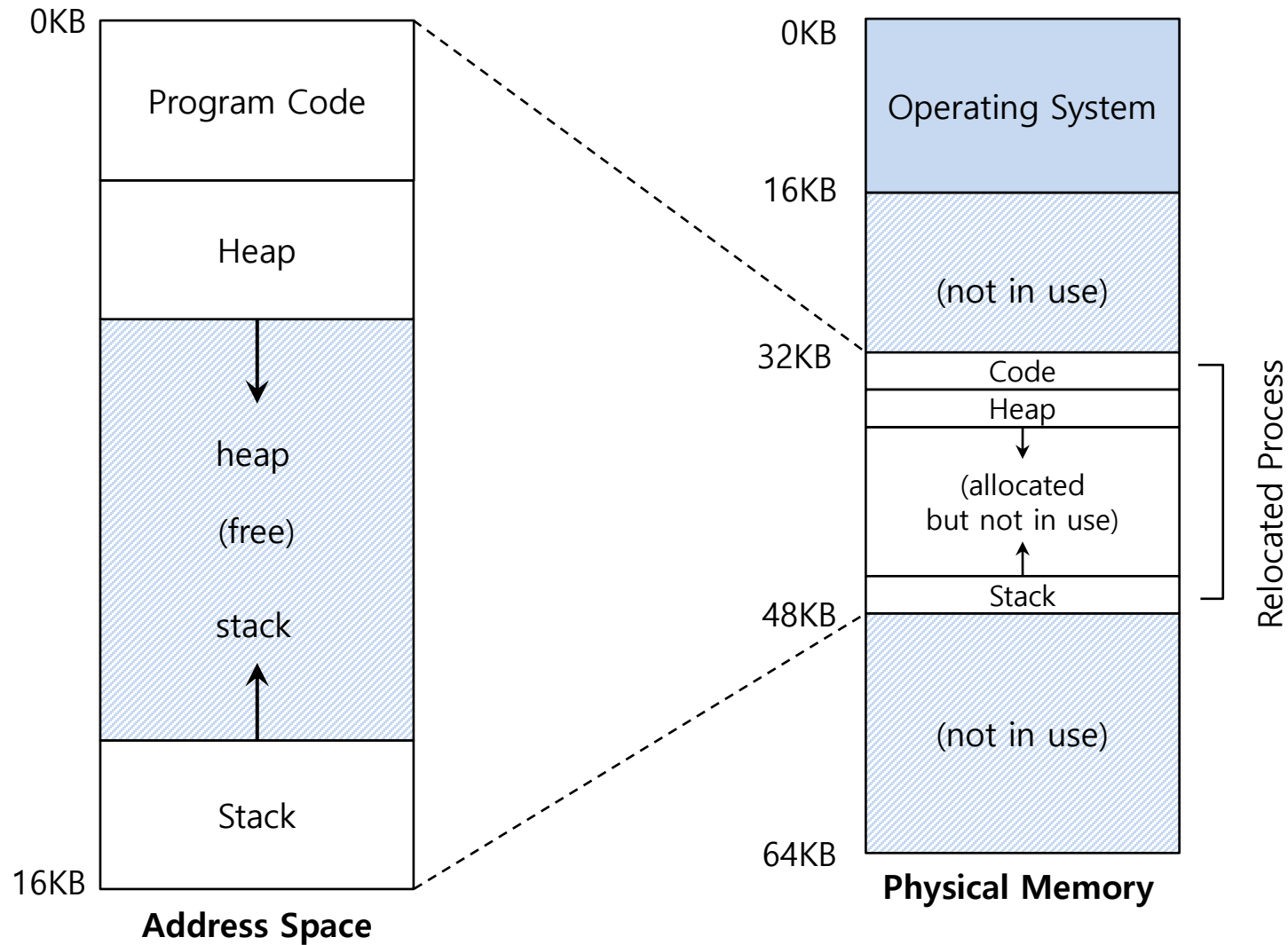
Dynamic (Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
 - ◆ Every virtual address must **not be greater than bound** and **negative**.

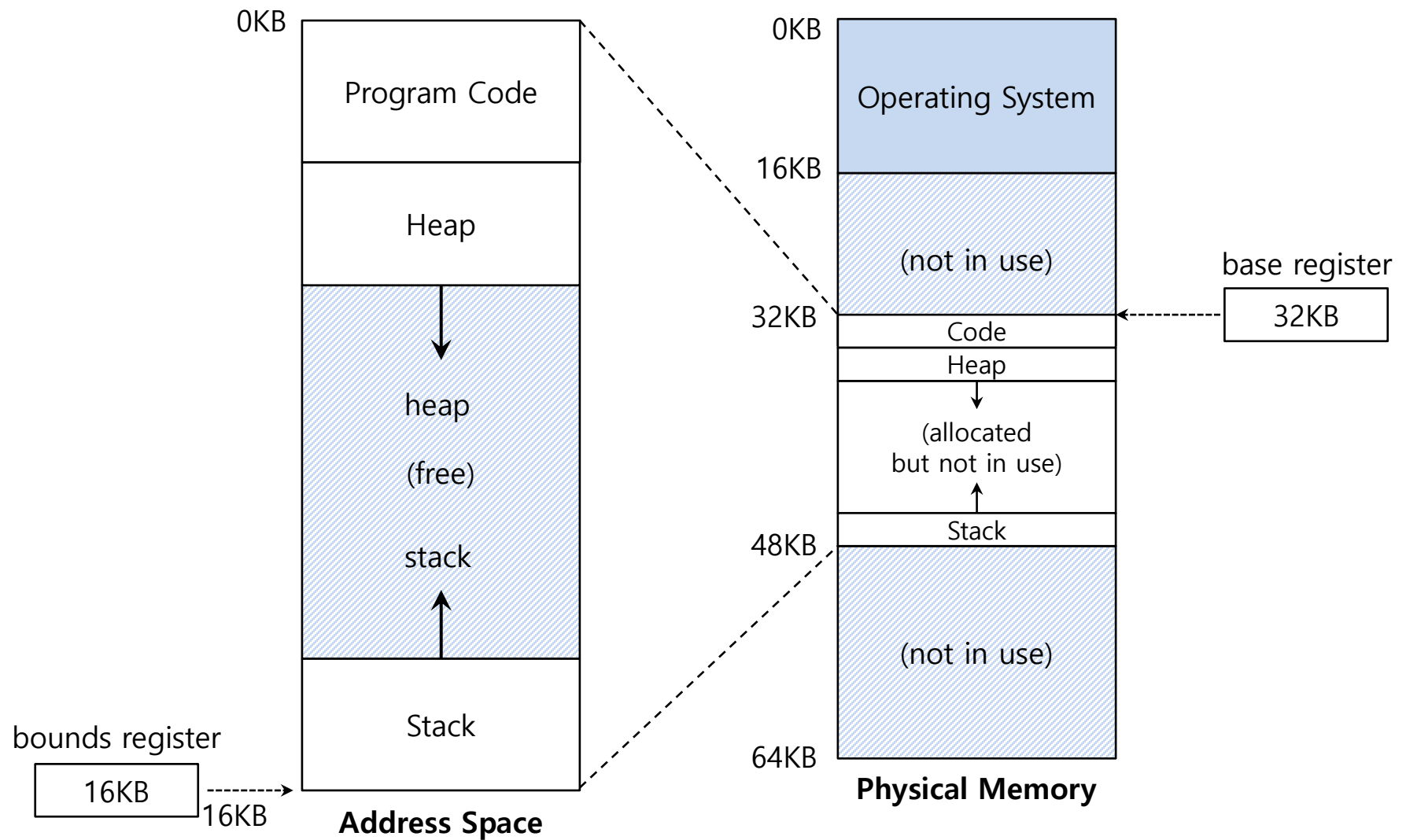
$$0 \leq \text{virtual address} < \text{bounds}$$

- ◆ Base: 32 KB.
- ◆ Virtual Address: 16 KB.
- ◆ Physical Address = 32 + 16 = 48 KB.

A Single Relocated Process



Base and Bounds Register



Relocation and Address Translation

128 : movl 0x0(%ebx), %eax

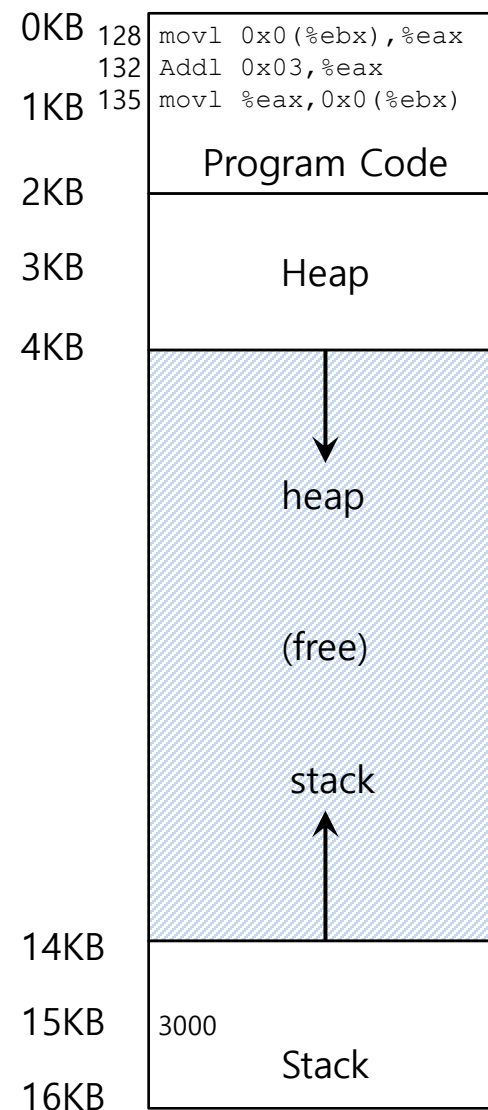
- ◆ **Fetch** instruction at address 128

$$32896 = 128 + 32KB(base)$$

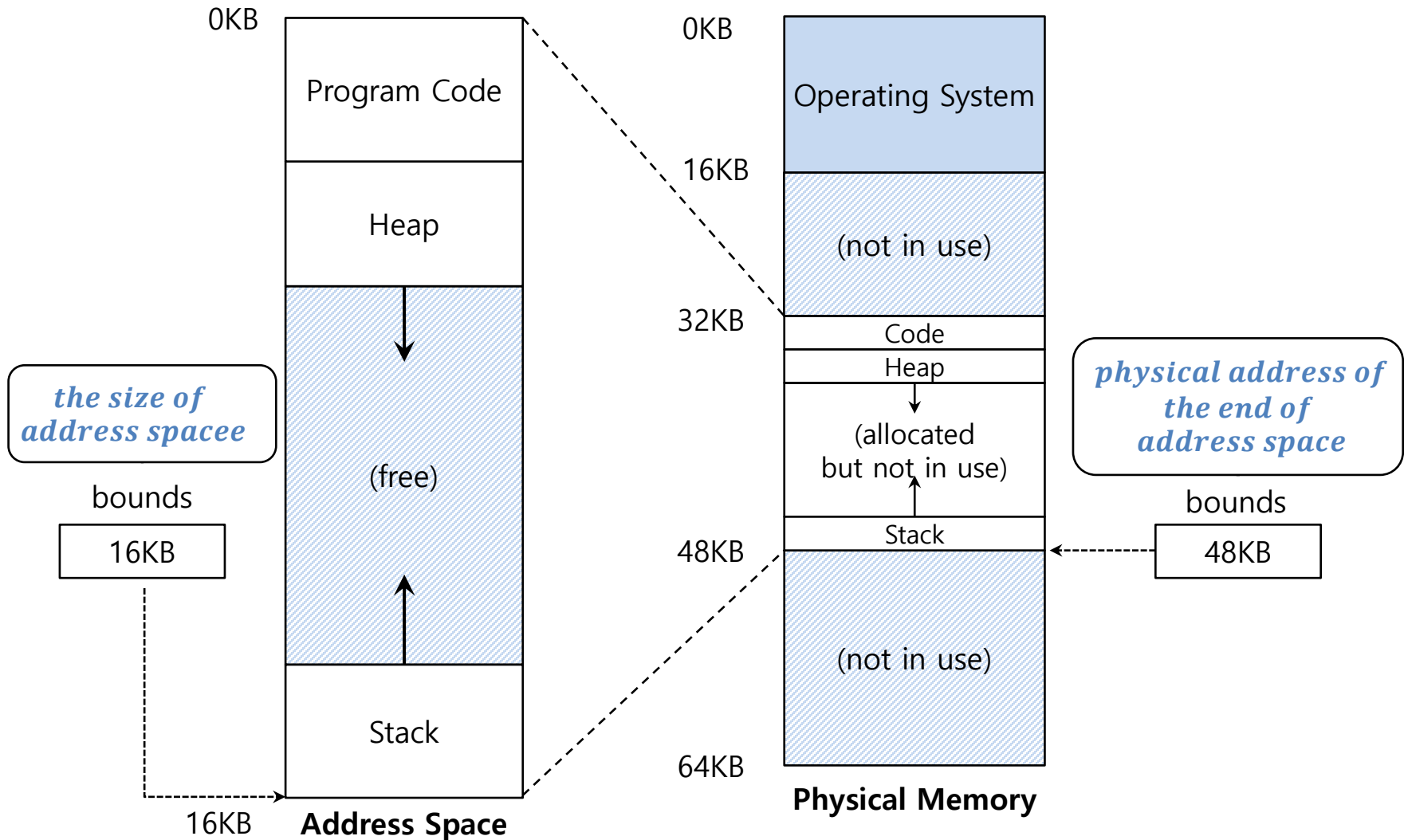
- ◆ **Execute** this instruction

- Load from address 15KB

$$47KB = 15KB + 32KB(base)$$



Two ways of Bounds Register



Memory Virtualizing – Hardware requirements

$$\text{physical address} = \text{virtual address} + \text{base}$$

$$\text{physical address} = \text{base} + \text{offset}$$

- ▣ The base and bounds register are part of the Memory Management Unit (MMU) of the CPU.

Memory Virtualizing – Hardware requirements

Hardware Requirements	Notes
Privileged mode	<i>Needed to prevent user-mode processes from executing privileged operations</i>
Base/bounds registers	<i>Need pair of registers per CPU to support address translation and bounds checks</i>
Ability to translate virtual addresses and check if within bounds	<i>Circuitry to do translations and check limits; in this case, quite simple</i>
Privileged instruction(s) to update base/bounds	<i>OS must be able to set these values before letting a user program run</i>
Privileged instruction(s) to register exception handlers	<i>OS must be able to tell hardware what code to run if exception occurs</i>
Ability to raise exceptions	<i>When processes try to access privileged instructions or out-of-bounds memory</i>

Figure 15.3: Dynamic Relocation: Hardware Requirements

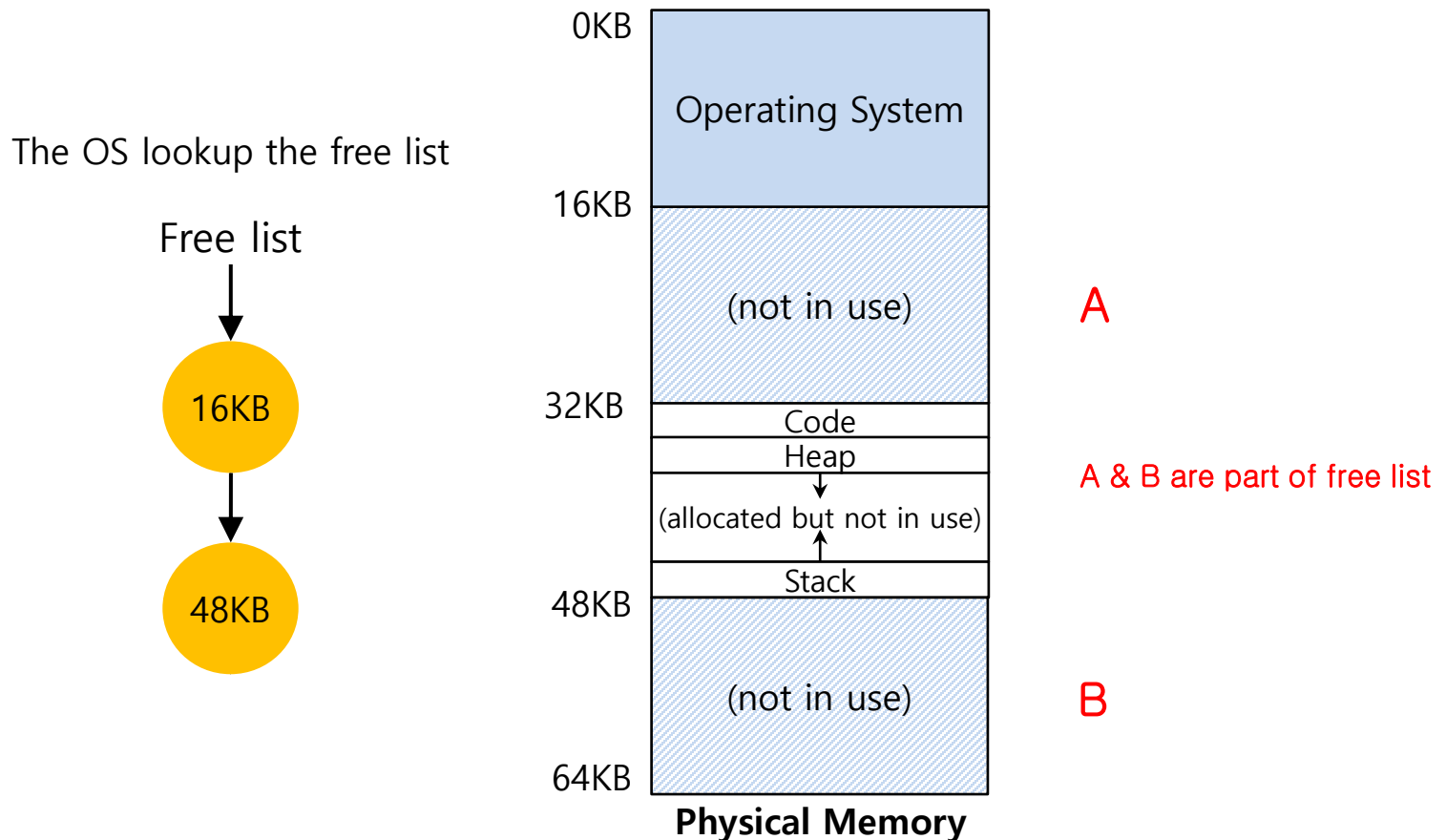
Exception handler

OS Requirements

OS Requirements	Notes
Memory management	<i>Need to allocate memory for new processes; Reclaim memory from terminated processes; Generally manage memory via free list</i>
Base/bounds management	<i>Must set base/bounds properly upon context switch</i>
Exception handling	<i>Code to run when exceptions arise; likely action is to terminate offending process</i>

OS Issues: When a Process Starts Running

- ▣ The OS must **find a room** for a new address space.
 - ◆ free list : A list of the range of the physical memory which are not in use.

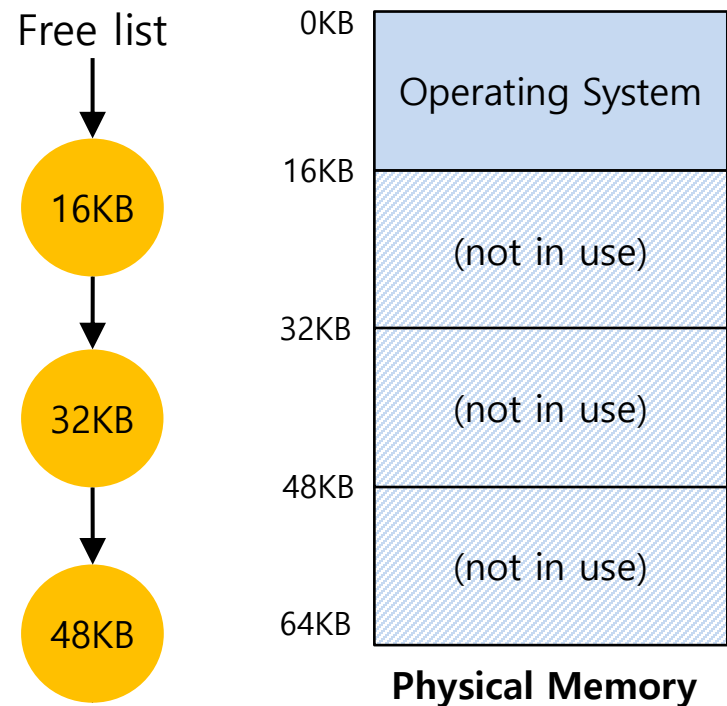
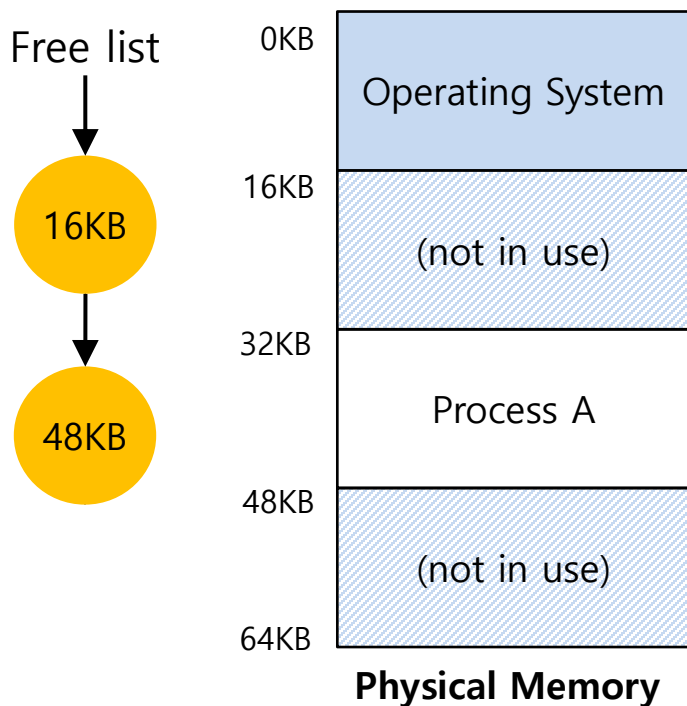


OS Issues for Memory Virtualizing

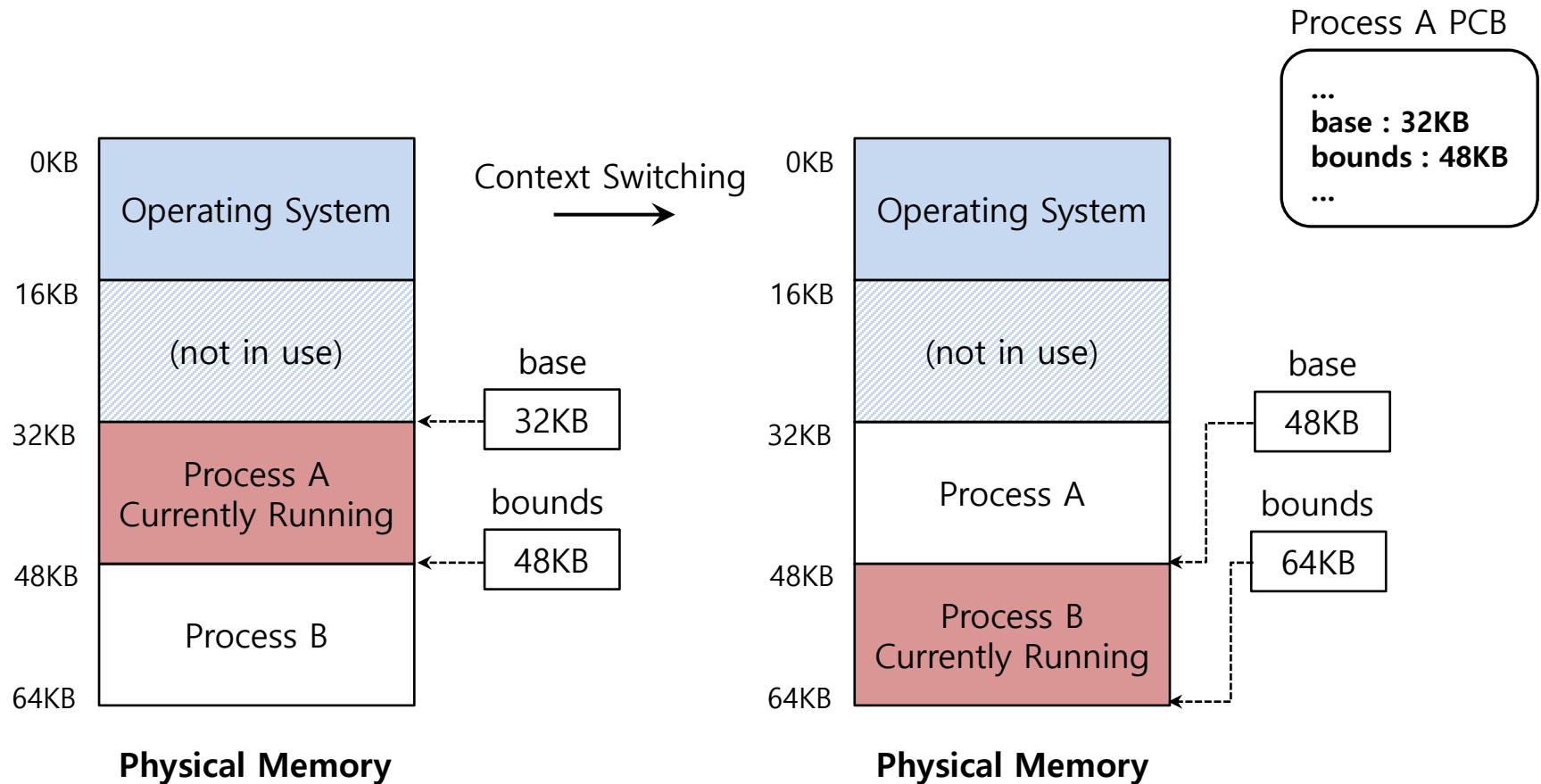
- ❑ The OS must **take action** to implement **base-and-bounds** approach.
- ❑ Three critical junctures:
 - ◆ When a process **starts running**:
 - Finding space for address space in physical memory
 - ◆ When a process is **terminated**:
 - Reclaiming the memory for use
 - ◆ When context **switch occurs**:
 - Saving and storing the base-and-bounds pair

OS Issues: When a Process Is Terminated

- Free up space: The OS must **put the memory back** on the free list.



- **Context Switch:** The OS must **save and restore** the base-and-bounds pair in the **process structure** or **process control block(PCB)**



- ❑ Disclaimer: This lecture slide set was initially developed for Operating System course in Computer Science Dept. at Hanyang University. This lecture slide set is for OSTEP book written by Remzi and Andrea at University of Wisconsin.